Without using function piping, this would be equivalent to:

```
String.replace(String.upcase("a string"), "A", "THE")
```

You'll encounter other operators and data types as you learn Elixir, but this covers the majority of what you'll use day-to-day. We'll cover a few other operators (such as string concatenation and combining lists) in Chapter 4, Pattern Matching Your Way to Success, on page 49.

If you run into something you don't understand, review the Kernel module documentation.[1] It has information about types, operators, and additional best practice considerations.

The Kernel Module



Elixir operators are either implemented by the compiler or in a special module called Kernel. The Kernel module is automatically available to all Elixir code, so you never have to think about it.

Elixir, unlike Ruby, doesn't let you extend modules like Kernel. So you can't automatically include functions in all modules.

Now, let's move on to modules and functions.

Module and Function Basics

Elixir is a functional language, so—as you would expect—the work of an Elixir application is done in functions. Functions take input data, process it according to the function's code, and then return a result. Modules are containers of functions that you have grouped together. They are mainly used as an organizational tool.

We are talking about modules and functions at the same time because they go hand-in-hand with each other. A module doesn't serve a purpose without functions, and functions need a module to exist in.

The main purpose of modules is to hold functions, but they actually do more than this. Later in this chapter, we'll look at additional syntax that's available to modules.

We'll cover two types of functions in this chapter: named functions and anonymous functions. Named functions are the most commonly used functions in an Elixir app (over 90% of a typical Elixir app). Especially as you start out, lean toward using named functions instead of anonymous functions.

1. https://hexdocs.pm/elixir/1.14/Kernel.html